

Mizan.ai

Reconciliation Engine & Data Pipeline — Findings

Comparator/reconciliation matching engine, the RAG offline ingestion pipeline, the shared document-extraction module, SQLite, and Postgres — what was found, what was fixed, and current live state.

Verification note: sections 1 covers work done directly in this session, verified live against the real API throughout. Sections 2–5 combine that history with fresh checks run today against the actual running containers (row counts, schemas, service names) rather than relying on memory alone.

1. Comparator — reconciliation matching engine

The matching engine (`features/comparator/matching.py`) runs five passes, each narrower in scope or weaker in evidence than the last. Every finding below was confirmed with a live test through the real API, not just code inspection, and every fix has a regression test.

1.1 The five passes

Pass	What it does	Output
1. Reference match	Exact reference match, unique on both sides. Amount-verified — if the reference matches but the amounts disagree beyond tolerance, it becomes an exception instead of a clean match	Clean match, or <code>amount_mismatch</code>
2. Unique amount match	Exact amount match, unique on both sides, for whatever Pass 1 left	Clean match
3. Fuzzy match	Amount-within-tolerance + date-within-proximity, for what's left	<code>fuzzy_match</code> / <code>ambiguous</code> / <code>unmatched</code>
4. Group match	2–4 leftover items on one side summing to one on the other — only when a real connecting signal exists (date proximity, shared description word, or a sequential reference number)	<code>group_match</code>
5. Sum match (manual)	Same shape as Pass 4, but with NO signal requirement at all — a wide date ceiling is the only bound. Manually triggered only, after a full human review of Passes 1–4	<code>sum_match</code>

Only Passes 1–2 ever produce a clean, auto-accepted match. Everything from Pass 3 onward always goes to a human for confirm/dismiss/unresolved — nothing past Pass 2 is ever auto-confirmed.

1.2 Real bugs found and fixed

All of these were caught by direct testing (often against files the user built specifically to probe a suspected weak spot), not by code review alone.

Bug	Root cause	Fix
XLSX header misread as data	Real exports (SAP ledgers, bank statements) prepend 1–3 title/metadata rows before the real header; the extractor always used pandas' default <code>header=0</code>	Auto-detect the header row: the first row with ≥ 2 non-blank cells, skipping title rows with only one wide cell
Amount column matched to a running balance	The generic amount-column synonym list included “balance”, which coexists with real Debit/Credit columns on ledger exports and means something different (a running total, not the transaction amount)	Removed “balance”/“value” from the synonym list; Debit/Credit now checked before falling back to a generic amount column
Silent order-dependent pick (Pass 3)	Fuzzy matching used a pool that shrank as earlier rows claimed candidates greedily — whichever row came first in the uploaded file's order silently won a shared candidate, with no record the loser was ever a contender	Candidates computed for every row against the SAME full, unmutated pool; a candidate claimed by more than one row is now flagged ambiguous for all of them

Bug	Root cause	Fix
Contested-candidate cards allowed a double-claim	Two different review items could both list the same candidate; confirming both independently was accepted by the backend with no cross-check	<code>/confirm</code> now rejects the whole request (422) if more than one confirmed item selected the same candidate row
Stale “unmatched” rows survived a later resolution	A row left “unmatched” in one round could get a real, confirmed home in a later round (e.g. a Pass 5 grouping) — but the original placeholder review item was never removed, so the final report showed the same row as both resolved and missing	<code>drop_stale_unmatched_items()</code> removes an “unmatched” item once its row is settled by any other confirmed item
AI narration described confirmed items as missing	Given correct, unambiguous context (including an explicit “status: confirmed”), the summarization model still described contested/grouped/resolved items as missing from the other side	Prompt rewritten to state explicitly that kind alone doesn't mean missing, and that a confirmed status always overrides what the description text might suggest — verified fixed via live re-test, not assumed

1.3 Deliberate design choices worth recording

- **No blind combinatorial search, ever.** Passes 4 and 5 both cap group size at 2–4 and both reject a target with more than one equally-valid grouping rather than guess between them. Pass 4 also requires the candidates to already share a real signal before their amounts are even summed — verified directly: two unrelated transactions that happen to sum to a third are correctly left unmatched, because neither ever enters the candidate pool in the first place.
- **The VAT-inclusive/exclusive diagnostic is display-only.** When an `amount_mismatch`'s gap is within ± 0.5 percentage points of exactly 15%, the report/UI relabels it as a possible VAT-recording difference. This never changes match/no-match — the tolerance check that decides a mismatch runs first and is untouched.
- **The leftover pool for Pass 5 has a precise, tested rule.** Confirmed rows are excluded; dismissed or still-unresolved rows return to the pool (a wrong pairing doesn't mean the underlying row is gone); and if a row is both settled via one item and still listed by a different, wrong item, settled wins.
- **Comparator has no persistence layer at all** (confirmed against the live Postgres schema — see §5). Every endpoint is stateless; the entire matched/reviewed state travels in the request/response body. A dismissed item today has no way to be recognized automatically next month.
- **One vendor/customer relationship per run, strictly.** No batch endpoint exists for reconciling multiple relationships in one pass.

2. RAG offline ingestion pipeline

Location: `features/explainer/offline/` (`extract.py`, `chunker.py`, `embed.py`, `references.py`, `table_descriptions.py`, `ingest.py`), run as a manual/monthly batch job (`services/offline_data_ingestion/`) — GPU required, not something triggered casually. Confirmed still structured this way as of today.

2.1 What it does

- PDF sources go through Docling for structured markdown + table extraction.
- XLSX/CSV sources are parsed directly with pandas, bypassing Docling entirely.
- Prose is chunked by article number (with a fallback for non-article documents like hub-overview pages) and embedded with a local BGE-M3 model; results are upserted into Qdrant.
- Tables get a separate, deterministic (non-AI) placeholder description embedded as a distinct chunk kind (`table_pointer`), while the real row data is stored in SQLite — the online retrieval path resolves a `table_pointer` hit into full row data at query time.

2.2 Real bugs found and fixed (prior sessions, still in effect)

Bug	Fix
Word-form article numbers (“ARTICLE FORTY-THREE”) weren't recognized by the digit-only heading regex	Added an English word-to-number normalizer, used by both the heading detector and the citation detector
Flat-text-only chunk boundaries missed real documents using different heading conventions	Boundaries now come from Docling's markdown headings first, with an ALL-CAPS heuristic to tell a real heading from a Docling rendering artifact
One document (SOCPA) had only 2 of 38 real articles promoted to markdown headings by Docling — the rest sat as inline text and got swallowed into one giant chunk	Hybrid detector: markdown headings + a flat-text scan for colon-terminated “Article N:” mentions outside heading spans
Non-article documents (hub overviews, decimal-numbered guidelines) lost 100% of their content silently — zero chunks, zero errors logged	Added a fallback chunker (heading-based where present, else paragraph groups) — recovered hundreds of previously invisible chunks across most of the corpus
Point IDs for a long article's split parts didn't include a part index — each split piece silently overwrote the previous one in Qdrant on upsert, so only the LAST part of any long article survived	Added <code>part_index</code> to the stable point-ID computation; affected the entire corpus, required a full wipe-and-rebuild
<code>insert_table_rows()</code> had no dedup — a retry after a partial failure left 3x duplicate rows in 8 tables	Delete existing rows for that <code>table_ref</code> before inserting — re-ingesting a document is now safe and idempotent

2.3 Shared code with Comparator/Calculator — an important cross-cutting fact

`features/common/document_extraction.py` (used by Calculator and Comparator for every uploaded PDF/XLSX/CSV) does not duplicate extraction logic — its `extract_pdf()` and `extract_xlsx_or_csv()` both delegate directly to `features/explainer/offline/extract.py`'s own functions, confirmed by reading the import statements directly. This means the header-row auto-detection fix made for Comparator this session (§1.2) also transitively improves the RAG ingestion pipeline's own XLSX/CSV parsing quality, and vice versa — the two pipelines are not independent.

3. SQLite — knowledge-base table storage

A named Docker volume (know_vat_zatca_sqlite), holding two tables: tables_registry (one row per table, with document_type/version/jurisdiction metadata) and table_rows (the actual row data, keyed by table_ref). Shared by the offline ingestion job and the online rag-online service.

3.1 The major finding: a real, systemic data-loss bug

Investigating a user-reported wrong answer (a VAT late-payment-penalty question) traced back to table_rows/tables_registry being completely empty — repo-wide, not just for the one table in question — while the separate Qdrant volume still held full, correct data. The SQLite volume was evidently wiped or recreated at some point after the original ingestion run while the Qdrant volume survived untouched. Every table-based answer in the knowledge base was silently returning nothing real.

3.2 What was safely recoverable, and what wasn't

Tables sourced from a directly-readable CSV/XLSX file (still present in docs/rag_ingestion_data/) can be deterministically re-inserted by replaying the exact same extraction logic used at original ingestion time — no GPU or Docling needed. Tables originally extracted from a PDF via Docling's table-structure recognition cannot be safely reconstructed this way; guessing at their content would mean fabricating a record, which the RAG system's own prompt rules explicitly forbid.

Live state, verified today:

table_ref	Document	Rows	Source
vat_penalties_(taxation_violation_fines)_table_0	VAT Penalties (Taxation Violation Fines)	10	CSV — backfilled
electronic_invoice_data_dictionary_table_0	Electronic Invoice Data Dictionary (Introduction sheet)	26	XLSX — backfilled
electronic_invoice_data_dictionary_table_1	Electronic Invoice Data Dictionary (Data dictionary sheet)	152	XLSX — backfilled

3 of 245 total table_pointer chunks registered in Qdrant are backed by real SQLite data today. The remaining 242 (XML Implementation Standard × 2 versions, Security Features Implementation Standard, Detailed E-Invoicing Guideline, Guideline for Zakat Documents, Zakat Collection Overview, VAT Implementing Regulations and its Amendments) are PDF-sourced and still empty — a known, deliberately deferred gap, not an oversight. Closing it requires re-running the real GPU-based offline ingestion pipeline.

4. Qdrant — vector store

The `know_vat_n_zatca` collection currently holds **1,243 points** (verified live today), split between prose chunks (article text) and `table_pointer` chunks (the placeholder description for each table, resolved against SQLite at query time — see §3). Qdrant was the volume that survived the SQLite data-loss incident intact, which is precisely what made the bug in §3.1 detectable: citations kept showing up correctly, but the actual row data behind them didn't.

5. Postgres

One database (`mizan_db`), four schemas, confirmed live today by querying the running container directly rather than assuming from memory:

Schema.table	Purpose	Rows (live)
<code>auth.users</code>	User accounts	12
<code>chatbot.user_profiles</code>	Chatbot-specific user profile data	—
<code>chatbot.user_activity_log</code>	Chatbot activity logging	—
<code>work.customer_memories</code>	Calculator: searchable filing-history memory (“what did I do for this customer last time”)	2
<code>work.filing_extractions</code>	Calculator: stored extraction results per filing	3
<code>work.user_filing_notes</code>	Calculator: filing notes, backing the past-filings Q&A endpoint	14
<code>work.user_feedback</code>	Contact-page feedback submissions	1
<code>public.checkpoints / checkpoint_blobs / checkpoint_writes / checkpoint_migrations</code>	LangGraph's own Postgres checkpointer — conversation state for the chatbot and the VAT/ZATCA assistant	917 (checkpoints)
<code>public.store / store_migrations</code>	LangGraph's cross-thread store tables	—

No comparator schema exists anywhere — confirms §1.3's finding that Comparator has zero persistence today; every reconciliation run's state lives only in the browser session and the request/response bodies of its own API calls.

What this confirms across the whole system

- Auth, chatbot memory, and Calculator's filing history are all real, live, persisted data in Postgres.
- Comparator is the one major feature with no database-backed memory at all — a deliberate, documented scope boundary, not a bug, but worth knowing before assuming any reconciliation state survives past the current browser session.
- The RAG knowledge base's real content lives in two places that must stay in sync (Qdrant for retrieval, SQLite for table row data) — the §3.1 incident is the concrete proof of what happens when they drift apart, and is worth remembering as a reason to treat that pair as one logical unit operationally, not two independent volumes.

6. Key decisions made along the way

Not every choice in §1 was purely a bug fix — several were explicit design decisions, made collaboratively, sometimes after considering and rejecting an alternative. Recorded here so the reasoning isn't lost.

6.1 Requirements specified directly by the user (not proposed by Claude)

- Pass 5 (sum match) must run **only after** a full human review of Passes 1–4's own findings — never automatically, never in parallel with that review. This is why it's a separate, manually-triggered endpoint rather than folded into the same `reconcile()` call as Pass 4.
- The leftover-pool rule for Pass 5 was specified precisely up front: confirmed items are excluded entirely; dismissed items return their rows to the pool (a wrong pairing doesn't mean the row itself is gone); unresolved items also stay in the pool; and if a row is claimed by one confirmed item while a different, wrong item still lists it, the confirmed claim wins.
- Pass 5 must be surfaced via a manual frontend button ("Check for more possible groupings"), not run automatically — an explicit change from how the original design proposal was framed.
- Contested-candidate cards needed a third explicit option beyond Confirm/Dismiss: "Not sure yet — leave unresolved, follow up later", for when more information is needed from the counterparty before deciding.

6.2 Alternatives considered and rejected

- **Real company/vendor names in report labels** (instead of generic "Our Records"/"Their Statement") — considered, then rejected. The real names sit in title rows that get intentionally discarded during header detection; reliably picking out which discarded line is a company name is a separate, fragile feature of its own, and guessing wrong would mean confidently mislabeling a side — worse than a safe generic label.
- **Per-card checkbox with JS-enforced mutual exclusion** for contested candidates — built first, then replaced. Once it became clear the real ask was "show these as one decision, not N separate ones," the design was reworked into a single combined card with native radio buttons, which gets mutual exclusion for free from the browser instead of needing hand-written cross-card unchecking logic.
- **Auto-unchecking the single-candidate checkbox when "Dismiss" is chosen** — flagged as a minor UI inconsistency (checked-but-dismissed is a reachable, slightly odd state), but confirmed harmless via a live test (the backend ignores a dismissed item's candidate selection entirely) and left as-is rather than adding complexity for a cosmetic-only case.
- **Widening Pass 5's group-size cap beyond Pass 4's 2–4** — not done. Kept identical for consistency and because a bigger cap raises both combinatorial cost and coincidental-sum risk; flagged as revisitable if real data ever needs 5+ installments grouped together.
- **Detecting cross-target contention** in Passes 4 and 5 (two different targets each explainable by an overlapping candidate pool) — not built. This is a real, acknowledged gap in the same family as the bug that WAS fixed (order-dependent silent picks), but closing it fully starts to look like a proper bipartite-matching problem; flagged explicitly rather than silently shipped as if it were already handled.

6.3 Tunable values proposed, sanity-checked, and approved as proposed

Value	Chosen	Reasoning
VAT-gap detection tolerance	±0.5 percentage points around 15%	Real VAT-inclusive/exclusive pairs should land near-exactly on 15%; a small band absorbs line-item rounding without being loose enough to mislabel an unrelated discrepancy
Pass 5 date ceiling	90 days	Wide enough for realistic installment schedules; still finite enough to rule out combining transactions from obviously unrelated periods

Value	Chosen	Reasoning
Pass 4/5 group size	2–4 items	A bigger combination happening to sum correctly by pure chance is far likelier than a small one

Each of these was written up and reported back before implementation, as requested, rather than picked silently.

6.4 Still open

- Whether the exported report's own column labels (still “SAP/Odoo Export”/“Second Document”) should be updated to match the review screen's plain-language labels (“Our Records”/“Their Statement”) — flagged as an inconsistency, not yet resolved either way.

7. Process notes — tooling issues hit along the way

None of these are bugs in Mizan.ai itself — they're friction points in the local Windows development environment that cost real time before being diagnosed, worth keeping a record of.

- **Git Bash / MSYS path mangling:** `docker exec/docker cp` calls with a leading-slash Unix-style path (e.g. `/tmp/file.py`) were silently rewritten to a Windows path by the MSYS layer before reaching Docker, producing confusing “file not found” errors even though the file really was there. Fixed by setting `MSYS_NO_PATHCONV=1` on those specific commands.
- **Windows console encoding false alarms:** printing an em-dash or other non-ASCII character to a cp1252 Windows terminal repeatedly showed up as a corrupted-looking replacement glyph, which looked like real data corruption at first glance. Each time, a direct byte-level UTF-8 check on the actual stored file confirmed the underlying data was correct — it was purely a terminal-display artifact, not a real bug, but worth re-verifying every time rather than assuming.
- **Test file relocation:** a folder of hand-built test files moved/renamed between sessions (Windows' own duplicate-download naming), breaking a hardcoded script path with a plain “file not found” error until re-located with a directory listing.
- **A “no output” false alarm during the SQLite investigation:** an early query against the `table_rows` table produced no visible output at all, which briefly looked like a hang or a silently swallowed error. A trivial sanity check (a plain `print("hello world")` through the same `docker exec` path) confirmed the tooling itself was fine — the real cause turned out to be the actual finding in §3.1: the table genuinely had zero rows.